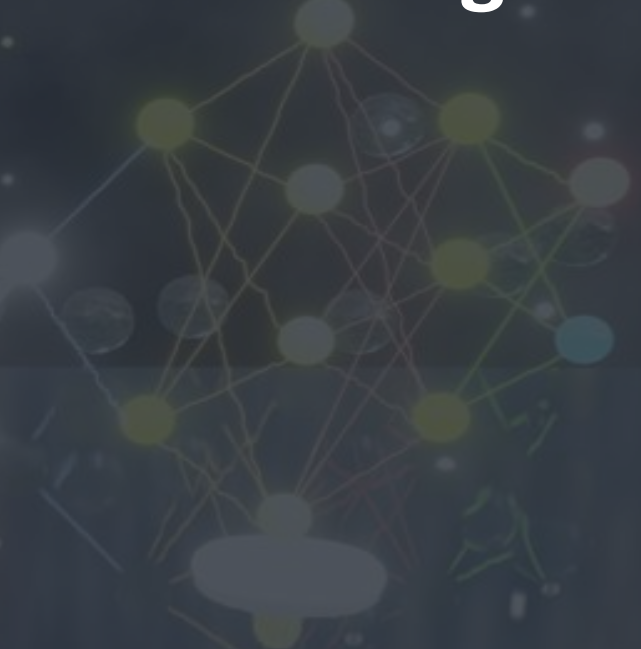
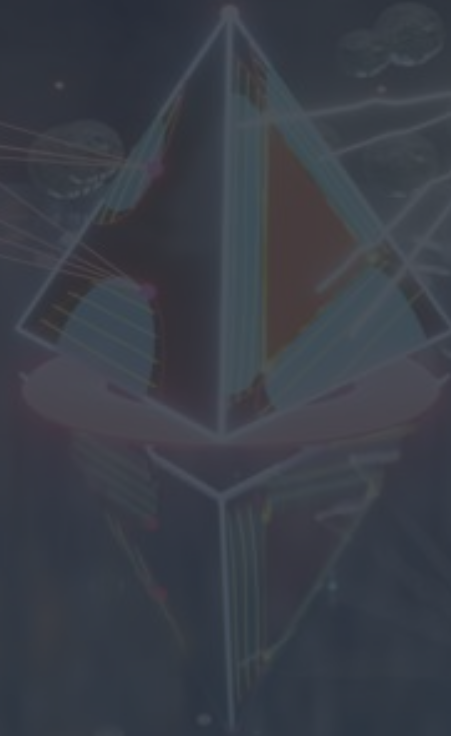
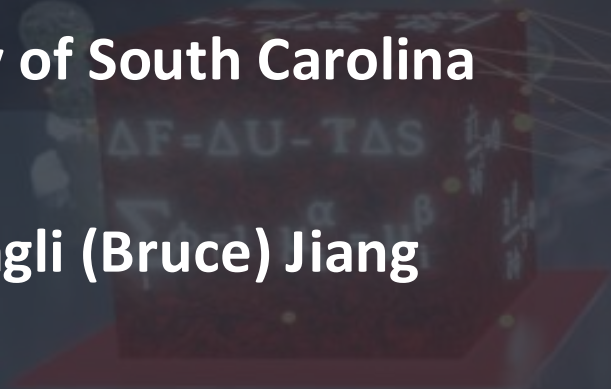


Feature Encoding: Preparing Mixed-Type Data for Machine Learning

Fall 2026

University of South Carolina

Prof. Shengli (Bruce) Jiang



From Lecture 10

Scaling & Feature Engineering

- *Linear scaling puts features on comparable scales.*
- *Nonlinear transformations can reduce skewness.*
- *Polynomial, interaction, and ratio features capture new relationships.*

In this lecture, you will learn

- *How to choose a categorical encoding method.*
- *How to handle missing values without data leakage.*

Numerical and Categorical Features

- Numerical features are measured quantities
 - Temperature = 523 K, 602 K
 - Pressure = 50 bar, 80 bar
 - Order and differences have physical meaning
- Categorical features are names or labels
 - Active material = Cu, In_2O_3
 - Support = ZnO, Al_2O_3
 - Labels identify groups. Their order has no meaning.

Are reactor IDs “523” and “072” numerical or categorical?

Categorical labels must be encoded without creating a false numerical order.

Dataset: Catalyst Materials in TheMeCat

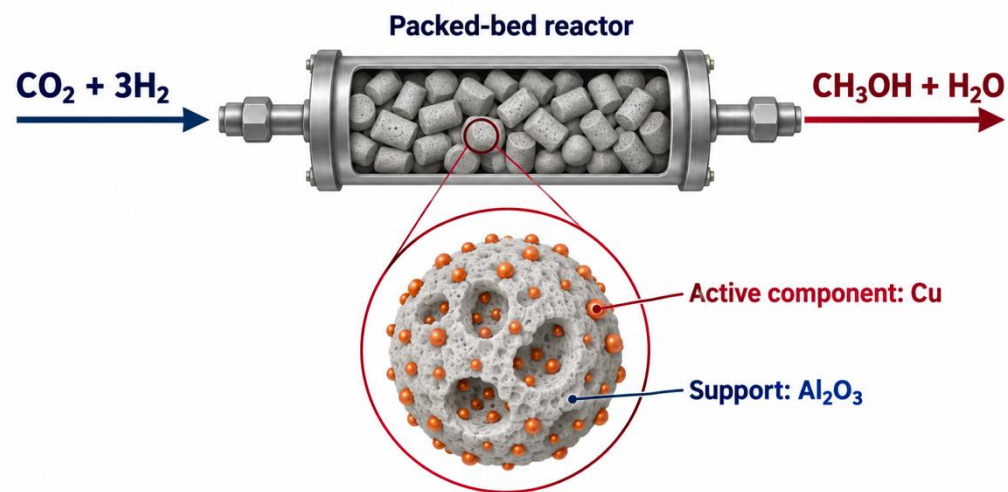
Data Descriptor | [Open access](#) | Published: 03 January 2026

TheMeCat: Dataset of Thermocatalytic Conversion of CO₂ to Methanol

[Árpád I. Toldy](#), [Prajwal Pisal](#), [Ondřej Krejčí](#), [Patrick Rinke](#) & [Annukka Santasalo-Aarnio](#) ✉

[Scientific Data](#) **13**, Article number: 165 (2026) | [Cite this article](#)

Each catalyst may contain active components and support components.



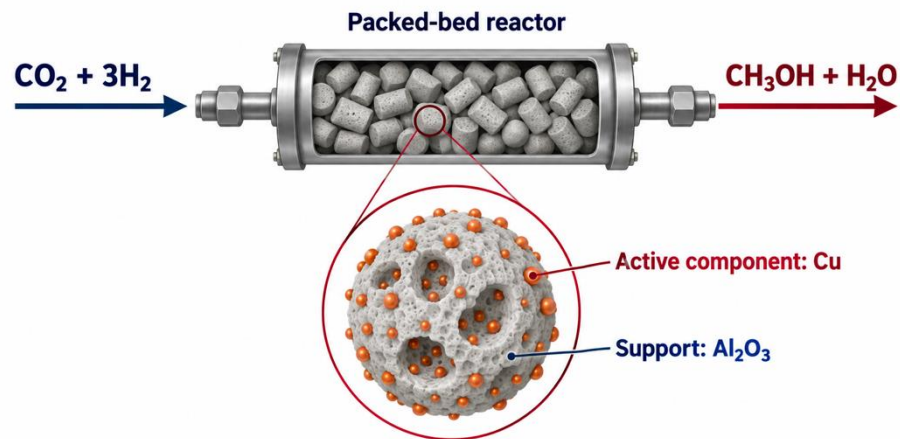
Active components

- Drive the CO₂ hydrogenation reaction
- Examples: Cu, In₂O₃, Pd, Pt

Support components

- Disperse and stabilize the active components
- Examples: ZnO, ZrO₂, Al₂O₃, SiO₂

Dataset: Catalyst Materials in TheMeCat



Features:

- Active material
- Support material
- Temperature
- Pressure
- Gas hourly space velocity (GHSV)
- H_2/CO_2 feed ratio

Target:

- Methanol produced per catalyst mass per hour

```
df = pd.read_csv("TheMeCat_v1.csv", na_values=["NA"])

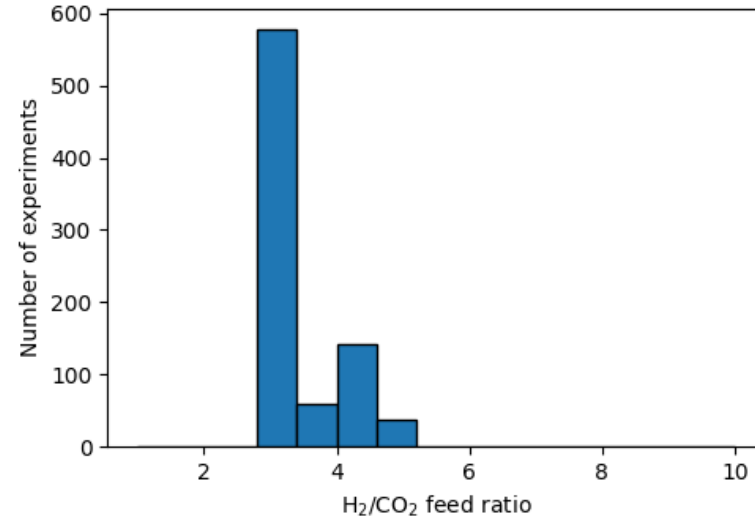
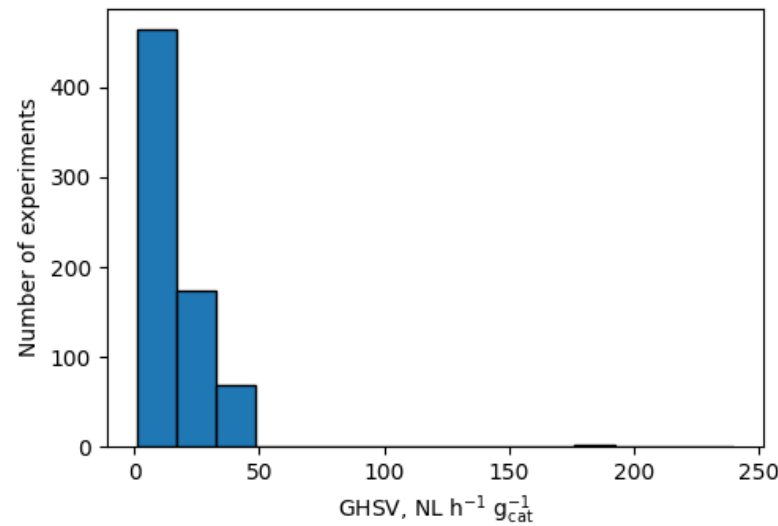
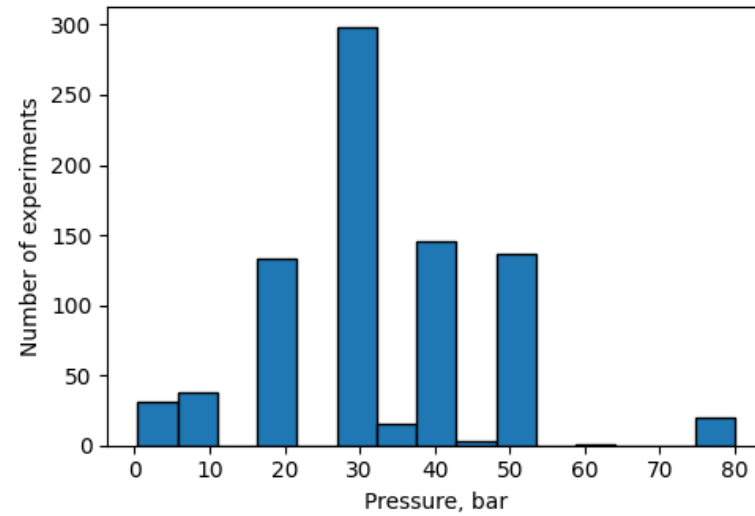
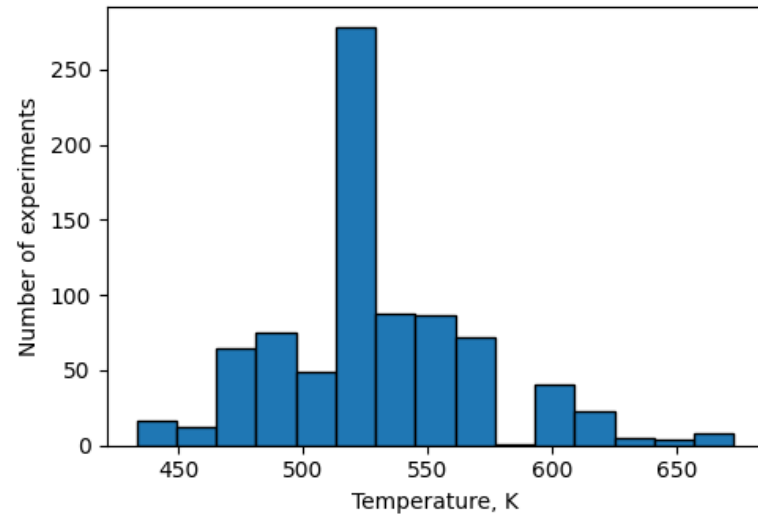
features = [
    "active_comp_1", "support_comp_1",
    "temperature_k", "pressure_bar",
    "GHSV_nlph_gcat", "pH2_pCO2_ratio"
]

target = "STY_g_per_gcath"

df[features + [target]].head()
```

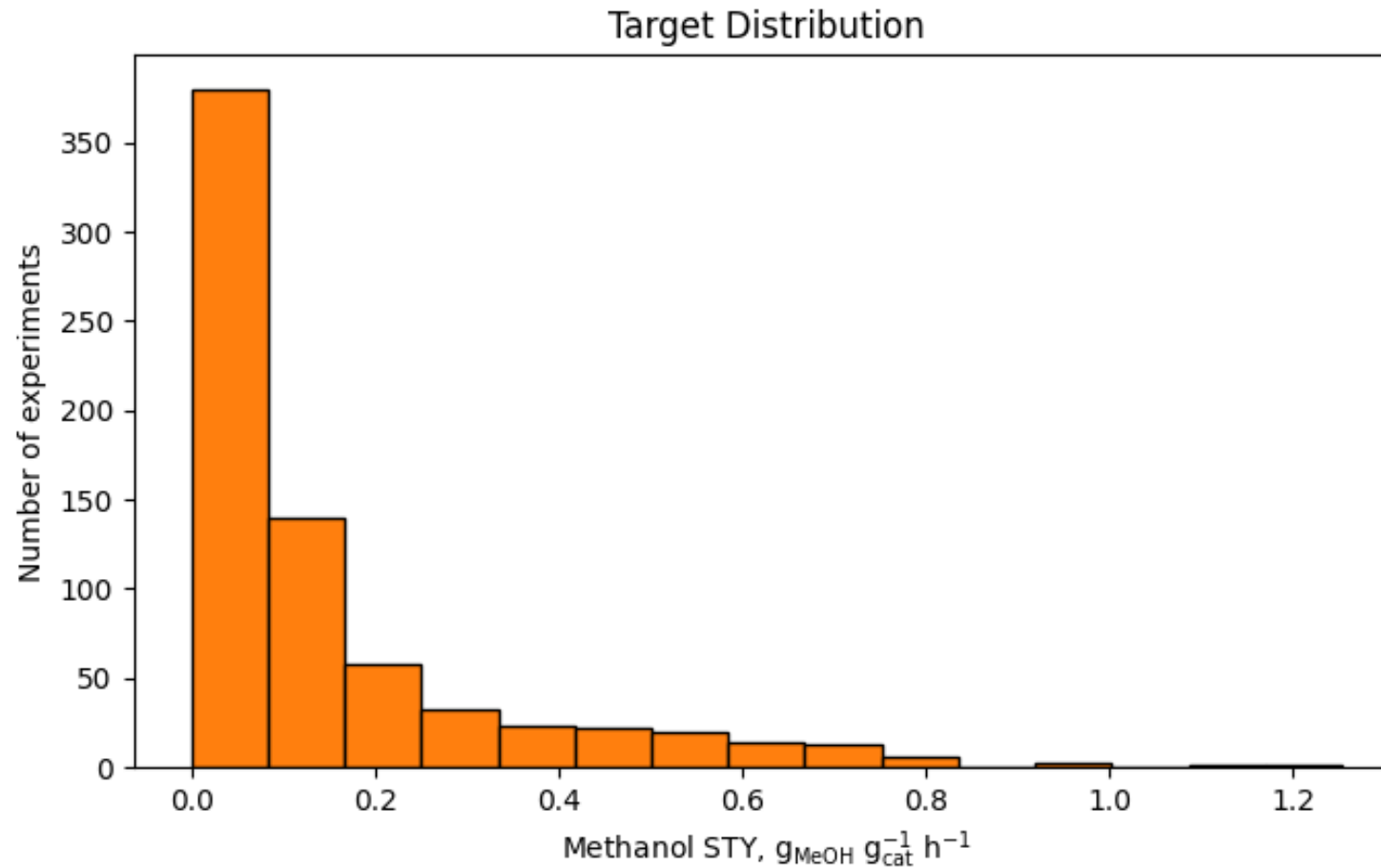
	active_comp_1	support_comp_1	temperature_k	pressure_bar	GHSV_nlph_gcat	pH2_pCO2_ratio	STY_g_per_gcath
0	Cu	Al2O3	513.0	30.0	4.4	3.0	0.1128
1	Cu	Al2O3	493.0	30.0	4.4	3.0	0.0944
2	Cu	Al2O3	473.0	30.0	4.4	3.0	0.0675
3	Cu	Al2O3	493.0	10.0	4.4	3.0	0.0462
4	Cu	Al2O3	473.0	10.0	4.4	3.0	0.0434

Numerical Features



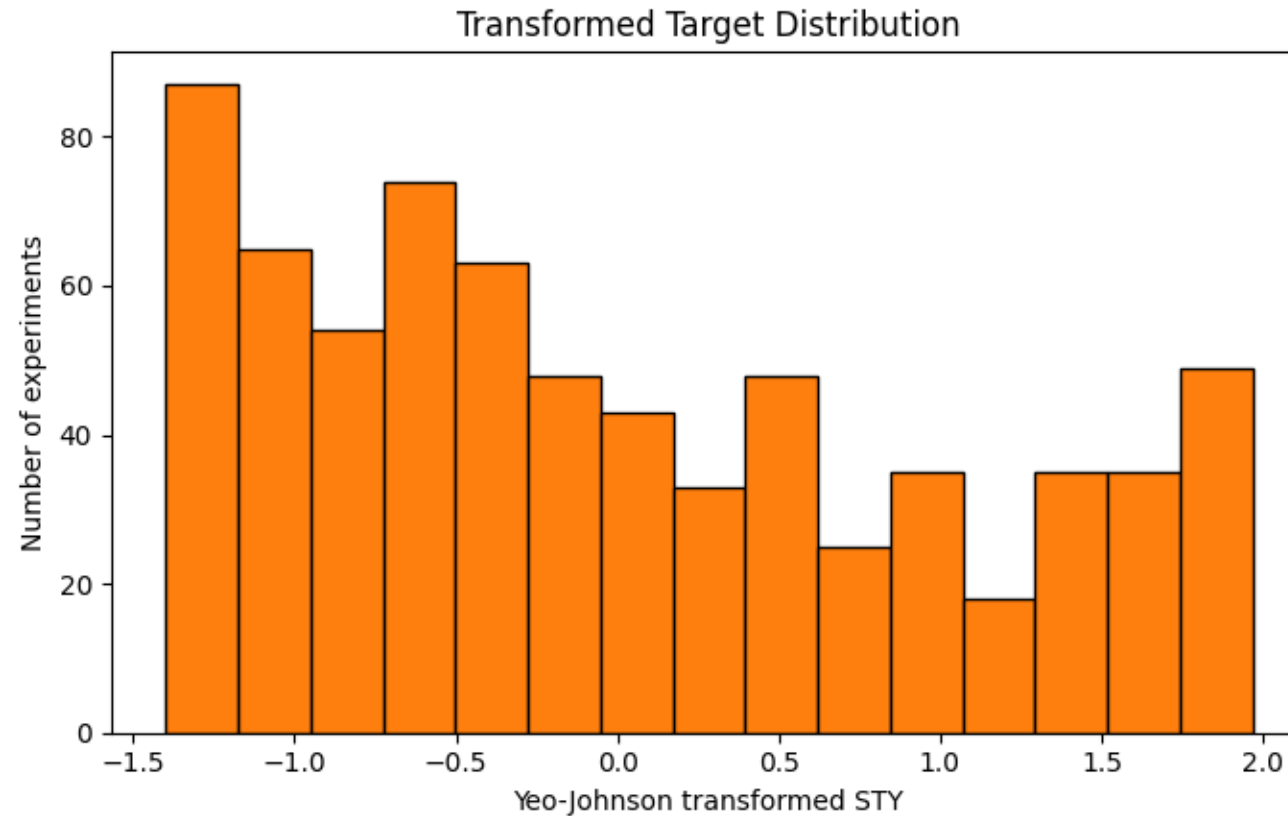
Numerical features have meaningful order and distance, even when their distributions differ.

Target



The target is strongly right-skewed and contains zeros.
Which transformation from last lecture should we use, and why?

Target after Yeo–Johnson Transformation



Yeo–Johnson can reduce skewness while retaining zero-valued observations.

Categorical Features

```
# Removes rows where every column is missing.
```

```
df = df.dropna(how="all").copy()
```

```
active_counts = df["active_comp_1"].fillna("Missing").value_counts()
```

```
support_counts = df["support_comp_1"].fillna("Missing").value_counts()
```

```
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
```

```
ax[0].bar(active_counts.index, active_counts.values, color="C0")
```

```
ax[0].set(xlabel="Active material", ylabel="Number of experiments",  
         title="Active Material Distribution")
```

```
ax[0].tick_params(axis="x", labelrotation=45)
```

```
ax[1].bar(support_counts.index, support_counts.values, color="C1")
```

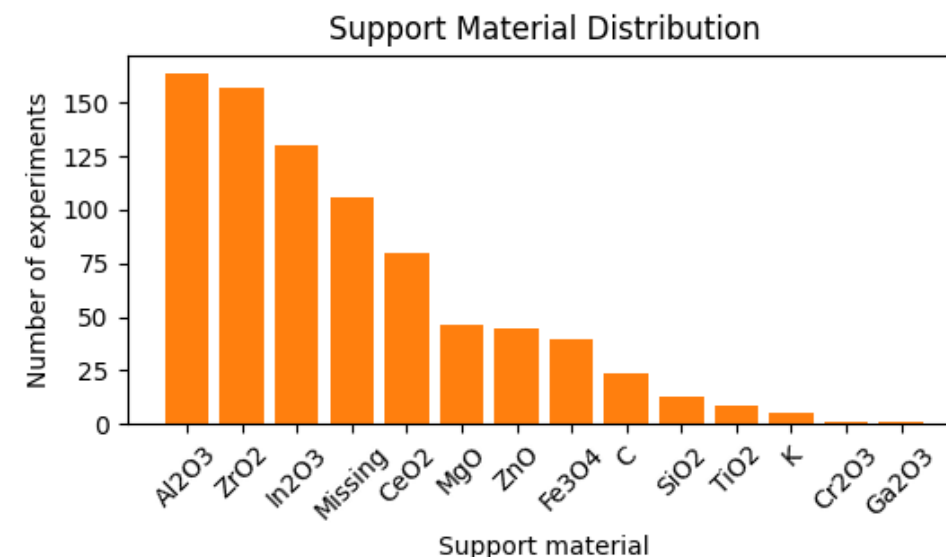
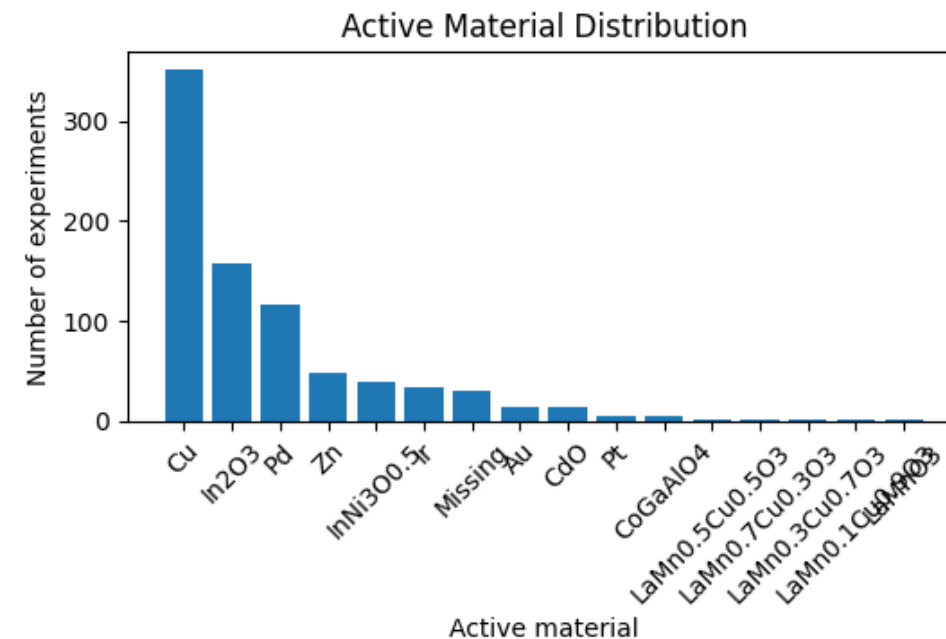
```
ax[1].set(xlabel="Support material", ylabel="Number of experiments",  
         title="Support Material Distribution")
```

```
ax[1].tick_params(axis="x", labelrotation=45)
```

```
fig.tight_layout()
```

```
plt.show()
```

Categorical features may be unevenly distributed or missing and must be encoded before modeling.



Why Categorical Features Need Encoding

Most models require numerical input.

Active material	Integer code
Cu	1
Pd	2
In ₂ O ₃	3

Does assigning 1, 2, and 3 create a meaningful order?

Arbitrary integer codes introduce relationships that the material labels do not have.

One-hot Encoding Principle

Create one binary column for each category.

```
from sklearn.preprocessing import OneHotEncoder
```

Active material	Encoding		
Cu	1	0	0
Pd	0	0	1
In2O3	0	1	0

Each observation receives a 1 for its category and 0 for every other category.

OneHotEncoder in scikit-learn

```
from sklearn.preprocessing import OneHotEncoder

example = pd.DataFrame({
    "active_comp_1": ["Cu", "Pd", "In203"]
})

encoder = OneHotEncoder()
encoded = encoder.fit_transform(example)
encoded = encoded.toarray()

encoded_df = pd.DataFrame(
    encoded,
    columns=encoder.get_feature_names_out()
)

display(encoded_df.astype(int))
```

	active_comp_1_Cu	active_comp_1_In203	active_comp_1_Pd
0	1	0	0
1	0	0	1
2	0	1	0

- By default, OneHotEncoder returns a **SciPy sparse matrix**. Here, we convert it to a dense array for display.
- With default **categories="auto"**, the encoder discovers the unique categories and orders strings alphabetically.

The encoder learns the categories and creates the corresponding binary features.

Encoding Multiple Features

```
categorical_features = [
    "active_comp_1",
    "support_comp_1"
]

X_cat = df[categorical_features]

encoder = OneHotEncoder(
    handle_unknown="ignore",
)

X_encoded = encoder.fit_transform(X_cat)
X_encoded = X_encoded.toarray()

print("Original shape:", X_cat.shape)
print("Encoded shape:", X_encoded.shape)
```

Original shape: (1124, 2)

Encoded shape: (1124, 30)

Why does the encoded dataset contain more columns?

Each categorical feature expands into one column for every category it contains.

```
encoder = OneHotEncoder(
    handle_unknown="ignore",
)

X_train_encoded = encoder.fit_transform(
    X_train[categorical_features]
)

X_train_encoded = X_train_encoded.toarray()

X_test_encoded = encoder.transform(
    X_test[categorical_features]
)

X_test_encoded = X_test_encoded.toarray()
```

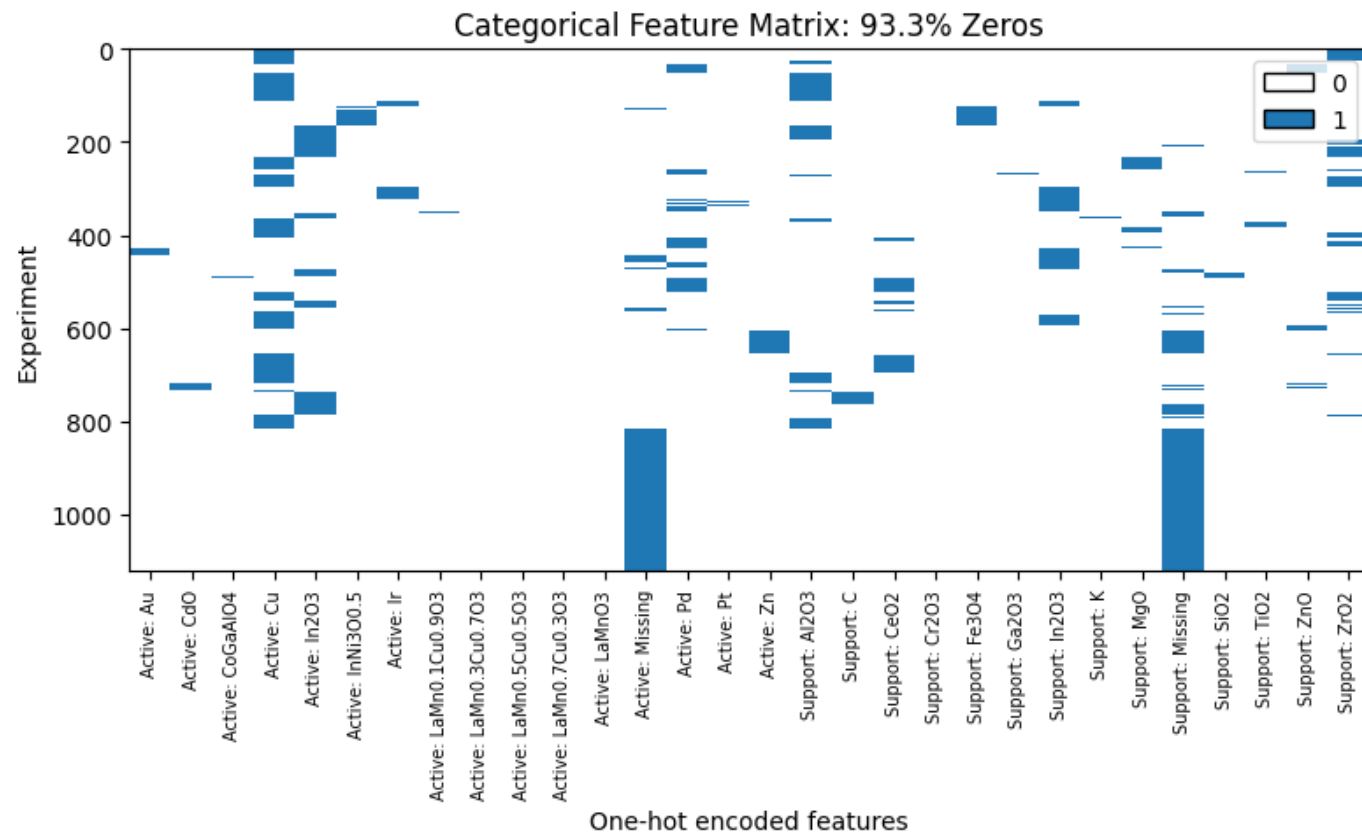
What happens when the test set contains a material absent from the training set?

With **handle_unknown="ignore"**, unseen categories produce **zeros**.

Limits of One-hot Encoding

A feature with k categories becomes k binary columns.

- TheMeCat contains 15 active-material categories and 13 support categories.
- These two features expand into 28 columns before handling missing values.
- Most encoded values are zero.
- Rare materials create columns with very few ones, giving the model little information to learn their effects.



One-hot encoding works best when the number of categories is limited and each category has enough observations.

Quiz

Experiment	Active material	Support material
1	Cu	Al ₂ O ₃
2	Pd	ZnO
3	Cu	ZnO
4	In ₂ O ₃	Al ₂ O ₃

1. How many columns will one-hot encoding create?
2. What is the encoded row for Experiment 2?
3. What percentage of each encoded row is zero?

Quiz

Experiment	Active material	Support material
1	Cu	Al ₂ O ₃
2	Pd	ZnO
3	Cu	ZnO
4	In ₂ O ₃	Al ₂ O ₃

1. How many columns will one-hot encoding create?
 - Active_Cu, Active_In₂O₃, Active_Pd, Support_Al₂O₃, Support_ZnO
2. What is the encoded row for Experiment 2?
 - 0, 0, 1, 0, 1
3. What percentage of each encoded row is zero?
 - There are 5 columns, and 60% of each row is zero.

Ordinal Encoding

Replace ordered categories with integers that preserve their ranking.

```
from sklearn.preprocessing import OrdinalEncoder
```

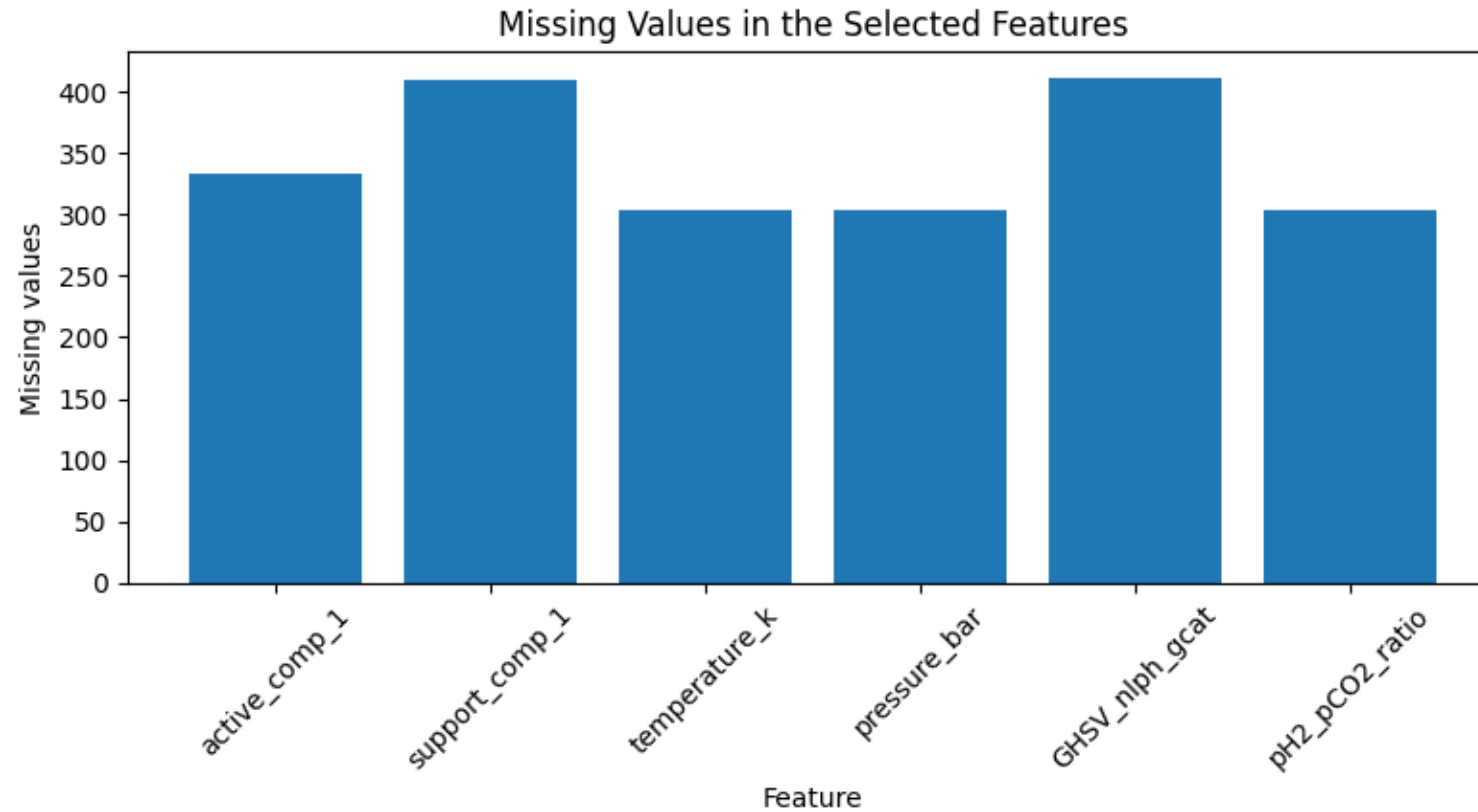
Feature	Encoded Feature
Low	0
Medium	1
High	2

Can we encode Cu, In₂O₃, and Pd as 0, 1, and 2?

Ordinal encoding is appropriate only when the categories have a meaningful order.

Let's Handle Missing Values

```
missing = model_df[features].isna().sum()
```



Do all missing values require the same treatment?

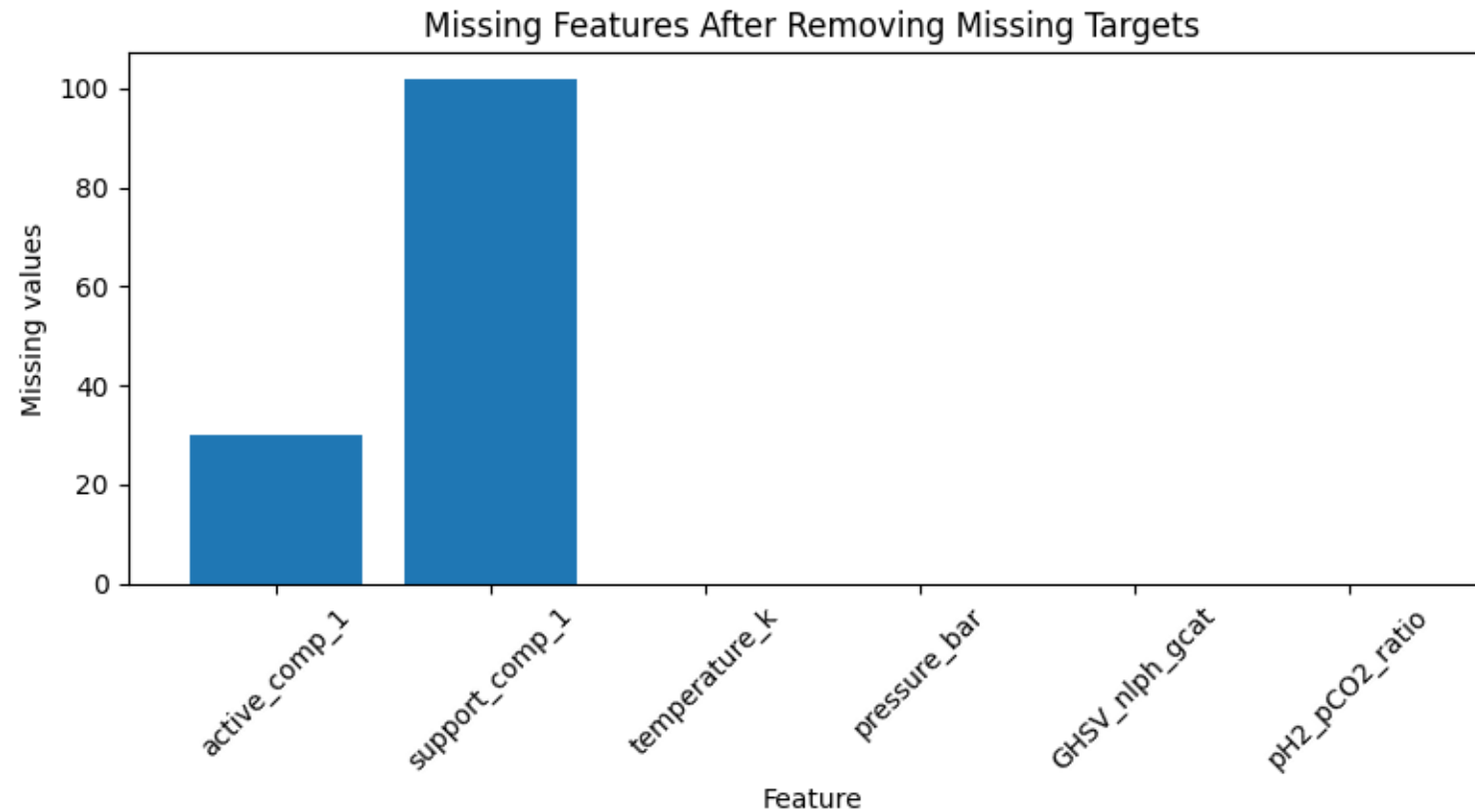
Remove or Impute?

Missing data	Treatment
Entirely blank row	Remove the row
Missing target	Remove from supervised training
Missing input feature	Impute when appropriate

```
model_df = df[features + [target]].dropna(subset=[target]).copy()
```

Training requires observed targets, while missing input features can often be imputed.

Missing Values After Target Filtering



**After keeping rows with observed target,
missing values remain only in active material and support material.**

Numerical Imputation

Replace a missing numerical value with a statistic learned from the training data.

```
from sklearn.impute import SimpleImputer

numeric_imputer = SimpleImputer(strategy="median")

temperature = df[["temperature_k"]]
temperature_imputed = numeric_imputer.fit_transform(temperature)
```

- **Mean** uses the arithmetic average.
- **Median** is less sensitive to extreme values.
- The imputed value comes from the training data.

Median imputation retains observations and limits the influence of extreme values.

Categorical Imputation

```
categorical_features = [  
    "active_comp_1",  
    "support_comp_1"  
]  
  
categorical_imputer = SimpleImputer(  
    strategy="constant",  
    fill_value="Missing"  
)  
  
X_cat_imputed = categorical_imputer.fit_transform(  
    model_df[categorical_features]  
)
```

- `strategy="most_frequent"` inserts the most common category.
- `strategy="constant"` creates an explicit missing category.

Should a missing support material automatically become Al2O3 because it is common?

Workflow: Leakage-Safe Preprocessing

1. Keep rows with observed targets

2. Train-validation-test split

3. Fit preprocessing on training data

Categorical: impute missing values,
then one-hot encode

Numerical: median imputation

**4. Apply the fitted preprocessing to
validation and test data**

**Learn imputation values and category levels from the training set,
then reuse them for validation and test data.**

Preprocessing with ColumnTransformer

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

categorical_features = ["active_comp_1", "support_comp_1"]

numerical_features = [
    "temperature_k", "pressure_bar",
    "GHSV_nlph_gcat", "pH2_pCO2_ratio"
]

features = categorical_features + numerical_features

X = model_df[features]
y = model_df[target]

# Development: 80%, test: 20%
X_dev, X_test, y_dev, y_test = train_test_split(
    X, y, test_size=0.20, random_state=26
)

# Training: 60%, validation: 20%, test: 20%
X_train, X_val, y_train, y_val = train_test_split(
    X_dev, y_dev, test_size=0.25, random_state=26
)
```

```
categorical_preprocessor = Pipeline([
    ("imputer", SimpleImputer(strategy="constant", fill_value="Missing")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("categorical", categorical_preprocessor, categorical_features),
    ("numerical", "passthrough", numerical_features)
])

X_train_prepared = preprocessor.fit_transform(X_train)
X_val_prepared = preprocessor.transform(X_val)
X_test_prepared = preprocessor.transform(X_test)

print("training:", X_train_prepared.shape)
print("validation:", X_val_prepared.shape)
print("test:", X_test_prepared.shape)

training: (426, 33)
validation: (143, 33)
test: (143, 33)
```



Pipeline: Preprocessing Steps in Sequence

A **Pipeline** applies each step in order and passes its output to the next step.

```
categorical_preprocessor = Pipeline([
    ("imputer", SimpleImputer(strategy="constant", fill_value="Missing")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])
```

- "imputer" and "encoder" are names assigned to the steps.
- **SimpleImputer** replaces missing labels with "Missing".
- **OneHotEncoder** receives the imputed labels and creates binary features.
- The order of the list determines the order of execution.

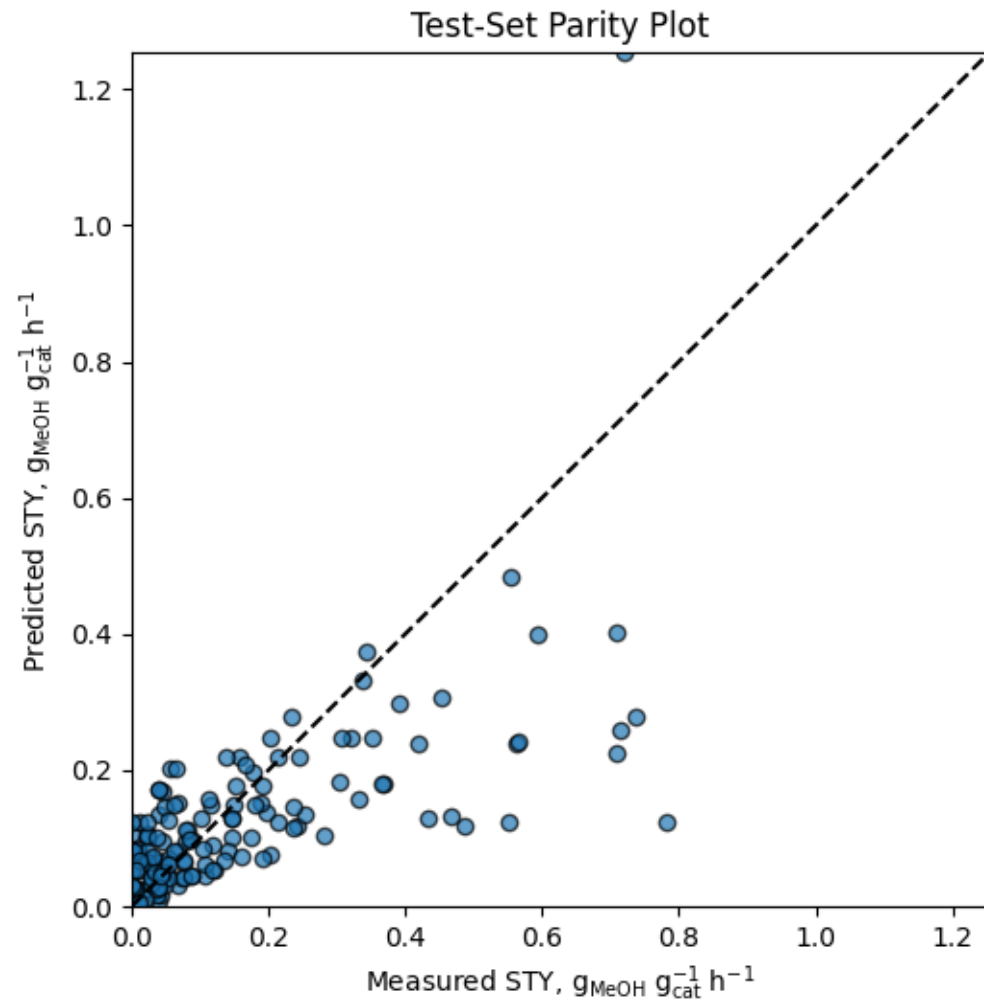
ColumnTransformer: Different Steps for Different Columns

A **ColumnTransformer** applies separate preprocessing to selected feature groups and combines the outputs.

```
preprocessor = ColumnTransformer([
    ("categorical", categorical_preprocessor, categorical_features),
    ("numerical", "passthrough", numerical_features)
])
```

- "categorical" and "numerical" name the two preprocessing branches.
- **categorical_preprocessor** applies imputation and one-hot encoding to the categorical features.
- "passthrough" keeps the numerical features unchanged.
- **ColumnTransformer** combines both branches into one feature matrix.

Prediction Example



Ridge Regression (alpha=1.0)

Validation RMSE: 0.1238

Validation R2: 0.4788

Test RMSE: 0.1426

Test R2: 0.4027

1. The training data contain Al_2O_3 and ZrO_2 , but the test data contain TiO_2 . What should we do?

- A. Fit the encoder again on the test data
- B. Remove the test experiment
- C. Use `OneHotEncoder(handle_unknown="ignore")`
- D. Encode TiO_2 as the largest number

Answer: C. The encoder learned from the training data can safely transform unseen categories without refitting.

1. Which quantities should be learned using only the training data?

- A. Imputation values only
- B. One-hot categories and Yeo–Johnson parameters only
- C. Model parameters only
- D. Imputation values, encoding categories, transformation parameters, and model parameters

Answer: D. Fit the complete preprocessing workflow on the training data, then apply it unchanged to validation and test data.

Interactive Python Time

`11_feature_encoding.ipynb`